

BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA

Facultad de Ciencias de la Computación

Programación Concurrente y Paralela

Práctica 2

Productor – Consumidor

(Primer parcial)

Equipo:

Islas Gómez Luis Antonio — 202372860
Ramírez Candia Alfredo — 202375947
Pérez Flores Julio César — 202247445
Navarro Soto Mario Alberto — 202264602

Docente: Hilda Castillo Zacatelco

11 de septiembre de 2026

Índice general

1	Introducción y objetivo	2
2	Marco teórico	2
3	Desarrollo	2
3.1	Estructura del programa	2
3.2	Candado atómico	3
3.3	Región crítica	3
4	Problemas de sincronización y condiciones de carrera	3
5	Solución planteada	4
6	Pruebas y resultados	5
7	Conclusiones	6

1 Introducción y objetivo

El problema productor–consumidor consiste en dos procesos que se comunican datos: el productor los genera y el consumidor los retira. A diferencia de la versión original, ahora *cada proceso tiene su propio buffer en memoria* (un arreglo privado) y la comunicación entre ambos se realiza mediante un archivo de texto que se emplea como una memoria compartida lógica: el productor copia ahí lo que quiere enviar y el consumidor lo copia a su propio buffer. El objetivo es implementarlo únicamente con `fork()` y archivos de texto, sin emplear tuberías, señales, `wait`, `exec`, semáforos, mutex, monitores, memoria compartida real ni colas de mensajes. La sincronización se resuelve con espera ocupada (ciclos vacíos) y retardos.

2 Marco teórico

- **Región crítica:** fragmento de código que accede a un recurso compartido y no debe ejecutarse simultáneamente por dos procesos.
- **Condición de carrera:** fallo producido cuando el resultado depende del orden no determinista en que los procesos acceden al recurso.
- **Exclusión mutua:** garantía de que solo un proceso está en la región crítica a la vez.
- **Espera ocupada:** el proceso que no puede avanzar repite un ciclo hasta que la condición cambia; se añaden pequeños retardos (`usleep`) para ceder tiempo de CPU.
- **Buffer privado:** arreglo en memoria que pertenece a un solo proceso; solo él lo lee y lo escribe, por lo que no requiere sincronización.
- **Canal (archivo como memoria compartida lógica):** archivo de texto que ambos procesos leen y escriben y que hace de buzón entre el productor y el consumidor.
- **COBEGIN / COEND:** notación que encierra bloques de código que pueden ejecutarse en paralelo.

3 Desarrollo

3.1 Estructura del programa

Se crea un proceso productor con `fork()`. El proceso padre actúa como consumidor y el hijo como productor. Cada proceso declara su propio arreglo `bufferPropio[BUFFER_SIZE]` en memoria; el productor acumula ahí lo que genera y el consumidor ahí lo que recibe. La comunicación se hace con el archivo `memoria.txt` (una línea por elemento), que se usa como memoria compartida lógica, y la exclusión mutua con `candado.txt`. El productor copia al final del canal y el consumidor extrae el primer elemento, por lo que se respeta el orden FIFO.

3.2 Candado atómico

La exclusión mutua se logra creando el archivo candado con la bandera `O_CREAT | O_EXCL`. El sistema operativo realiza esa creación de forma indivisible: si el archivo ya existe, la llamada falla con `EEXIST` y el proceso reintenta en un ciclo (espera ocupada). Al terminar, el candado se libera con `unlink()`. El candado protege únicamente el acceso al canal; los buffers privados no lo necesitan porque pertenecen a un solo proceso.

```
1 void adquirirCandado(){
2     while(open(ARCHIVO_CANDADO, O_CREAT | O_EXCL | O_WRONLY, 0600) ==
3         -1){
4         usleep(1000);
5     }
6 }
7 void liberarCandado(){
8     unlink(ARCHIVO_CANDADO);
9 }
```

3.3 Región crítica

El conteo del canal y su modificación se realizan *dentro* del candado, de modo que la verificación y la operación forman una sola acción atómica frente al otro proceso. Los buffers privados quedan fuera de la región crítica porque cada uno pertenece a un solo proceso.

```
1 /* Productor: solo el acceso al canal es region critica */
2 adquirirCandado();
3 if(contarItems() < BUFFER_SIZE && n > 0){
4     agregarItem(bufferPropio[0]);
5     liberarCandado();
6     /* ... */
7 } else {
8     liberarCandado();
9 }
```

4 Problemas de sincronización y condiciones de carrera

Al compartir recursos mediante archivos surgen los siguientes riesgos, que la solución presentada debe evitar:

1. **Condición de carrera en el acceso concurrente.** Si los dos procesos leen y modifican el archivo sin coordinarse, sus operaciones se intercalan y el resultado depende del orden de ejecución.
2. **Falta de atomicidad del candado.** Un candado construido leyendo y luego escribiendo el archivo deja una ventana entre ambas operaciones; ambos procesos podrían encontrar el candado libre y entrar a la vez. Por eso se requiere una primitiva indivisible.

3. **Verificación y modificación separadas.** Comprobar el conteo del buffer antes de acceder a él produce una condición *check-then-act*: entre la comprobación y la operación el buffer puede cambiar por el otro proceso.
4. **Pérdida de actualizaciones.** Como `quitarItem()` reescribe todo el archivo, un agregado que ocurra entre la lectura y la reescritura se perdería. Igualmente, el productor podría desbordar el canal o el consumidor extraer de un canal vacío.
5. **Desbordamiento de los buffers privados.** El productor no debe generar más allá de la capacidad de su arreglo, ni el consumidor recibir más allá de la capacidad del suyo; de lo contrario se sobrescribiría memoria válida.
6. **Canal lleno o vacío.** Si el canal está lleno el productor no puede enviar aunque su buffer privado tenga datos; si está vacío el consumidor no puede recibir. Estas condiciones deben verificarse siempre dentro de la región crítica.

5 Solución planteada

La solución se basa en tres ideas: cada proceso mantiene su propio buffer en memoria, el canal se implementa con un archivo de texto y el candado se toma mediante una operación atómica del sistema de archivos; toda verificación y modificación del canal se realiza dentro de la región crítica. El siguiente fragmento muestra el algoritmo implementado, encerrado entre COBEGIN y COEND para representar los dos procesos que corren en paralelo.

```
1 // Canal compartido: memoria.txt. Candado: candado.txt.
2 // Cada proceso tiene su propio buffer en memoria: bufferPropio[.
3
4 // COBEGIN
5 // Productor (proceso hijo)
6 int bufferPropio[BUFFER_SIZE];
7 int n = 0;
8 int numero = 1;
9 while(1){
10     // Produce en su buffer privado si hay espacio.
11     if(n < BUFFER_SIZE){
12         bufferPropio[n] = numero;
13         n++;
14         numero = (numero % 10) + 1;
15     }
16     // Envía al canal solo si hay espacio y tiene algo pendiente.
17     adquirirCandado();
18     if(contarItems() < BUFFER_SIZE && n > 0){
19         int enviado = bufferPropio[0];
20         agregarItem(enviado);
21         for(int i = 1; i < n; i++) bufferPropio[i-1] = bufferPropio[i];
22         n--;
23         liberarCandado();
```

```

24     printf("[Productor] envio: %d\n", enviado);
25 } else {
26     liberarCandado();
27 }
28 usleep(200000);
29 }
30
31 // Consumidor (proceso padre)
32 int bufferPropio[BUFFER_SIZE];
33 int n = 0;
34 while(1){
35     // Recibe del canal a su buffer privado si hay espacio.
36     adquirirCandado();
37     if(contarItems() > 0 && n < BUFFER_SIZE){
38         bufferPropio[n] = quitarItem();
39         n++;
40     }
41     liberarCandado();
42     // Consume de su buffer privado.
43     if(n > 0){
44         int item = bufferPropio[0];
45         for(int i = 1; i < n; i++) bufferPropio[i-1] = bufferPropio[i];
46         n--;
47         printf("\t\t[Consumidor] tomo: %d\n", item);
48     }
49     usleep(500000);
50 }
51 // COEND

```

Con esto, `adquirir/liberar` garantizan exclusión mutua sobre el canal y las verificaciones de lleno/vacío ya no compiten con el otro proceso, mientras que cada buffer privado se manipula sin candado.

6 Pruebas y resultados

Se compiló con `gcc ComePica.c -o bin/comepica -Wall -Wextra` sin errores y se ejecutó durante algunos segundos. La salida muestra que el canal nunca supera los 10 elementos, que ninguno de los buffers privados se desborda y que el consumidor solo retira cuando hay datos; el orden de retiro es FIFO.

```

1 [Productor] envio: 1
2 [Productor] envio: 2
3 [Productor] envio: 3
4     [Consumidor] tomo: 1
5 [Productor] envio: 4
6 ...
7     [Consumidor] tomo: 5

```

7 Conclusiones

Separar los buffers privados del medio de comunicación simplifica la sincronización: cada proceso manipula su arreglo sin candado y solo el acceso al archivo que hace de memoria compartida lógica requiere exclusión mutua. La principal enseñanza es que no basta con “bloquear” el acceso: separar la verificación de la operación deja abierta una condición de carrera. Colocar ambas acciones bajo el candado y usar una primitiva indivisible del sistema de archivos elimina las pérdidas de actualizaciones y respeta tanto la capacidad del canal como la de cada buffer privado.